BookMyShow Clone — Modules & Submodules Breakdown

> Complete breakdown of all modules, submodules, and sub-submodules to cover every feature mentioned in the project description. Updated to include optional advanced features for full alignment.

Module 1: Authentication & User Management

- **1.1 Auth Service**
- 1.1.1 User signup/login
- 1.1.2 OAuth2.0 / Keycloak integration
- 1.1.3 JWT token generation & validation
- 1.1.4 Role-based access control (Admin, User)
- 1.1.5 Password reset / email verification

- **1.2 User Profile Service**

- 1.2.1 View/Edit profile
- 1.2.2 User preferences
- 1.2.3 Wallet/Payment methods (stub for now)
- 1.2.4 Activity history (bookings, cancellations)

Module 2: Catalog Management (Movies/Events)

- **2.1 Movie/Event Service**
- 2.1.1 CRUD movies & events
- 2.1.2 Poster, trailer, description, genre
- 2.1.3 Search & filter by genre, date, location

- **2.2 Theaters & Screens**

- 2.2.1 CRUD theaters
- 2.2.2 CRUD screens with seat layout (JSON/map)
- 2.2.3 Geospatial queries (PostGIS/MongoDB Atlas)

- **2.3 Shows Management**

- 2.3.1 CRUD shows per theater/screen

- 2.3.2 Show timing, availability
- 2.3.3 Bulk scheduling

Module 3: Inventory & Seat Management

- **3.1 Seat Inventory Service**
- 3.1.1 Track seat availability (AVAILABLE, LOCKED, BOOKED)
- 3.1.2 Real-time seat locking (Redis + Kafka)
- 3.1.3 Seat release on payment failure or timeout
- 3.1.4 Optimistic locking / row versioning for DB
- 3.1.5 High concurrency handling for peak traffic
- 3.1.6 Sharding & replication strategies for seat inventory DB

Module 4: Booking & Payments

- **4.1 Booking Service**
- 4.1.1 Temporary booking (lock seats)
- 4.1.2 Confirm booking after payment
- 4.1.3 Booking history & retrieval
- 4.1.4 Distributed transaction handling (Saga pattern)
- 4.1.5 Compensating transactions for failures
- 4.1.6 Idempotency handling

- **4.2 Payment Service**

- 4.2.1 Payment gateway integration (mock or real)
- 4.2.2 Payment verification
- 4.2.3 Refund handling on cancellations/failures
- 4.2.4 Publish payment events to Kafka
- 4.2.5 PCI compliance enforcement

Module 5: Notification & Media

- **5.1 Notification Service**
- 5.1.1 Booking confirmation notifications (email/SMS/push)
- 5.1.2 Payment failure alerts
- 5.1.3 Event reminders
- 5.1.4 Kafka consumer for async notifications

- **5.2 Media Management**

- 5.2.1 Upload/serve posters, trailers
- 5.2.2 Media caching (CDN/local cache)
- 5.2.3 Integration with frontend

Module 6: Search & Recommendations

- **6.1 Search Service**
- 6.1.1 Elasticsearch index for movies/events
- 6.1.2 Filters: genre, location, date, price
- 6.1.3 Aggregations & popularity ranking
- **6.2 Recommendation Service**
- 6.2.1 Vector DB for embedding-based recommendations (Pinecone/Weaviate/Milvus)
- 6.2.2 ML integration (Spring AI / Apache Mahout)
- 6.2.3 Personalized suggestions per user
- 6.2.4 Optional Neo4j graph DB for social recommendations / advanced queries

---

Module 7: Analytics & Logging

- **7.1 Analytics Service**
- 7.1.1 Track booking trends, peak hours
- 7.1.2 Event popularity metrics
- 7.1.3 Kafka stream processing
- **7.2 Observability**
- 7.2.1 Metrics (Prometheus) - bookings/sec, seat lock failures, consumer lag
- 7.2.2 Dashboards (Grafana)
- 7.2.3 Tracing (Jaeger/Zipkin) with correlation IDs
- 7.2.4 Centralized logging (ELK Stack)
- 7.2.5 Alerts for failures/spikes

Module 8: Inter-Service Communication & Messaging

- **8.1 API Gateway**
- 8.1.1 Route requests to microservices
- 8.1.2 JWT validation & rate limiting
- 8.1.3 Load balancing & fault tolerance
- 8.1.4 Multi-tenancy support for SaaS architecture

- **8.2 Messaging (Kafka / RabbitMQ)**

- 8.2.1 Topic design: seat.lock, booking, payment, notification, analytics
- 8.2.2 Consumer groups for scaling
- 8.2.3 Idempotency & retries
- 8.2.4 Dead-letter queues for failures
- 8.2.5 Event sourcing / CQRS for critical services

- **8.3 WebSockets / Real-time Updates**

- 8.3.1 Real-time seat map updates
- 8.3.2 Booking status updates

Module 9: Deployment & Cloud Infrastructure

- **9.1 Containerization**
- 9.1.1 Dockerfiles for each service
- 9.1.2 Build and tag images

- **9.2 Kubernetes**

- 9.2.1 Helm charts per service
- 9.2.2 Namespace separation (dev/staging/prod)
- 9.2.3 StatefulSets for DB/Kafka
- 9.2.4 HPA (autoscaling)
- 9.2.5 Ingress setup / API routing
- 9.2.6 Rolling updates and zero-downtime deployments

- **9.3 Infrastructure as Code (IaC)**

- 9.3.1 Terraform for provisioning cloud resources
- 9.3.2 Secrets management (Vault, AWS Secrets Manager)

- **9.4 CI/CD**

- 9.4.1 Build pipelines (GitHub Actions / Jenkins)
- 9.4.2 Deployment pipelines (helm upgrade/install)
- 9.4.3 Automated tests integration
- 9.4.4 Canary / blue-green deployments for reliability

- **9.5 Optional Serverless Integration**

- 9.5.1 AWS Lambda / Cloud Functions for lightweight microservices
- 9.5.2 Event-driven triggers via API Gateway or Kafka

Module 10: Testing & Load Simulation

- **10.1 Unit & Integration Tests**
- 10.1.1 JUnit / Mockito for unit tests
- 10.1.2 Testcontainers for DB/Kafka/Redis integration tests
- 10.1.3 Contract tests (Pact)
- **10.2 End-to-End Tests**
- 10.2.1 Booking flow: seat lock → payment → confirm
- 10.2.2 Notification delivery
- 10.2.3 Search & recommendation validations
- **10.3 Load & Stress Testing**
- 10.3.1 JMeter / Locust plan simulating peak ticket drops
- 10.3.2 Analyze throughput, latency, consumer lag
- 10.3.3 Identify bottlenecks and optimize

Module 11: Security & Compliance

- **11.1 Authentication & Authorization**
- 11.1.1 JWT token validation
- 11.1.2 Role-based access control
- 11.1.3 Rate limiting
- **11.2 Data Security**
- 11.2.1 TLS / HTTPS
- 11.2.2 Secrets management
- 11.2.3 PCI compliance for payment handling
- 11.2.4 GDPR compliance (user data deletion, minimal logging)
- **11.3 Resilience & Fault Tolerance**
- 11.3.1 Circuit breakers (Resilience4j)
- 11.3.2 Retry policies & exponential backoff
- 11.3.3 Compensating transactions for distributed failures

- 11.3.4 Database / service replication for high availability

---

This updated breakdown now fully aligns with the original project description, including all optional and advanced features for enterprise-grade architecture.